ESCUELA POLITÉCNICA NACIONAL

NOMBRE: Nathaly Bravo

CURSO: GR1CC

In [ ]:

TALLER 03

Objetivo 1- Gráficar la funcion

- Vector xs
- Definir funcion ()

In [5]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import newton
```

In [9]:
```
xs = list(np.linspace(0, 5, 50))
```

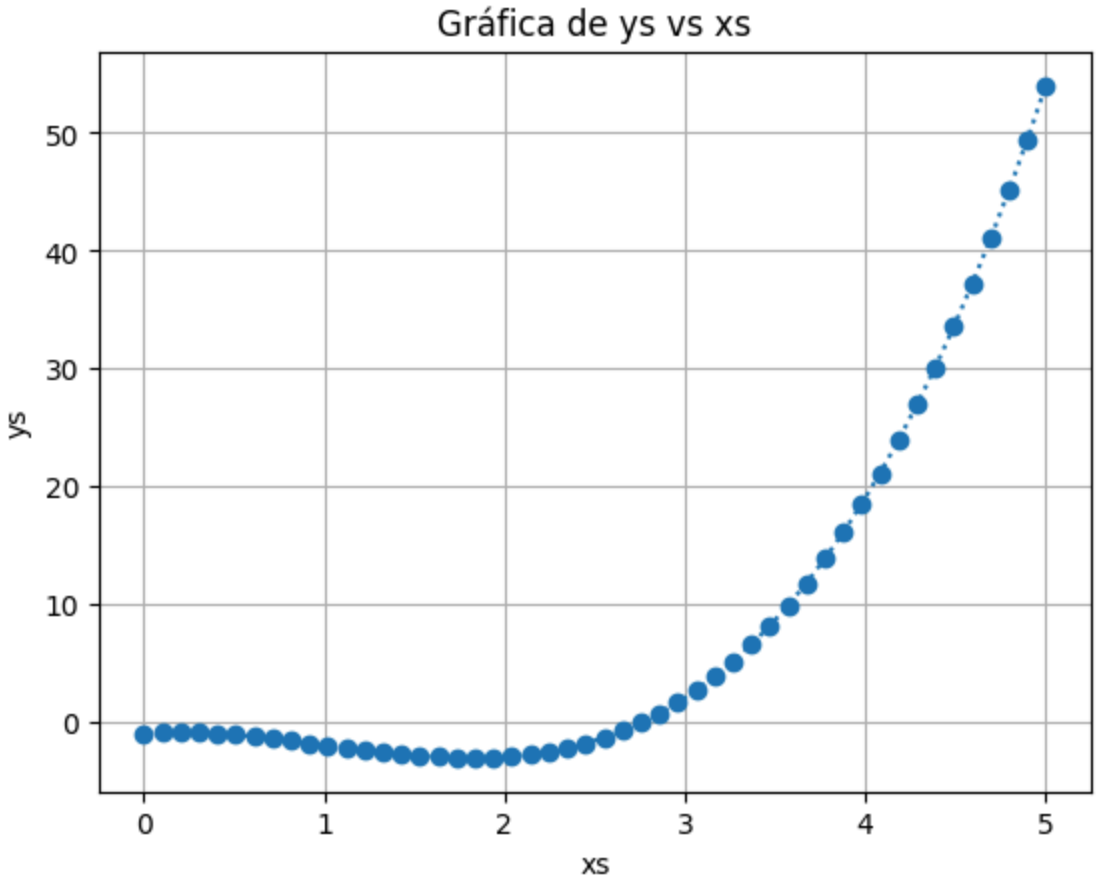
In [10]:
```
def f(x):
    _y = x**3 - 3 * x**2 + x - 1
    print(f"{x:2f}, {_y: .2f}")
    return _y
```

In [11]:
```
ys = [f(x) for x in xs]
```

```
0.000000, -1.00
0.102041, -0.93
0.204082, -0.91
0.306122, -0.95
0.408163, -1.02
0.510204, -1.14
0.612245, -1.28
0.714286, -1.45
0.816327, -1.64
0.918367, -1.84
1.020408, -2.04
1.122449, -2.24
1.224490, -2.44
1.326531, -2.62
1.428571, -2.78
1.530612, -2.91
1.632653, -3.01
1.734694, -3.07
1.836735, -3.09
1.938776, -3.05
2.040816, -2.95
2.142857, -2.79
2.244898, -2.56
2.346939, -2.25
2.448980, -1.86
2.551020, -1.37
2.653061, -0.79
2.755102, -0.10
2.857143,  0.69
2.959184,  1.60
3.061224,  2.63
3.163265,  3.80
3.265306,  5.09
3.367347,  6.53
3.469388,  8.12
3.571429,  9.86
3.673469,  11.76
3.775510,  13.83
3.877551,  16.07
3.979592,  18.49
4.081633,  21.10
4.183673,  23.90
4.285714,  26.90
4.387755,  30.11
4.489796,  33.52
4.591837,  37.16
4.693878,  41.01
4.795918,  45.10
4.897959,  49.43
5.000000,  54.00
```

In [12]:
```
import matplotlib.pyplot as plt

plt.plot(xs, ys, "o:")
plt.xlabel("xs")
plt.ylabel("ys")
```

```
plt.title("Gráfica de ys vs xs")
plt.grid(True)
plt.show()
```



Objetivo 2

- Leer documentacion de scipy.optimize.newton
- Verficiar scipy esta isntalado
- Definir derivada de la funcion
- Implementar para nuestro caso

In [13]:
```
from scipy.optimize import newton

newton(f, x0=100, fprime=lambda x: 3 * x**2 - 6 * x + 1)
```

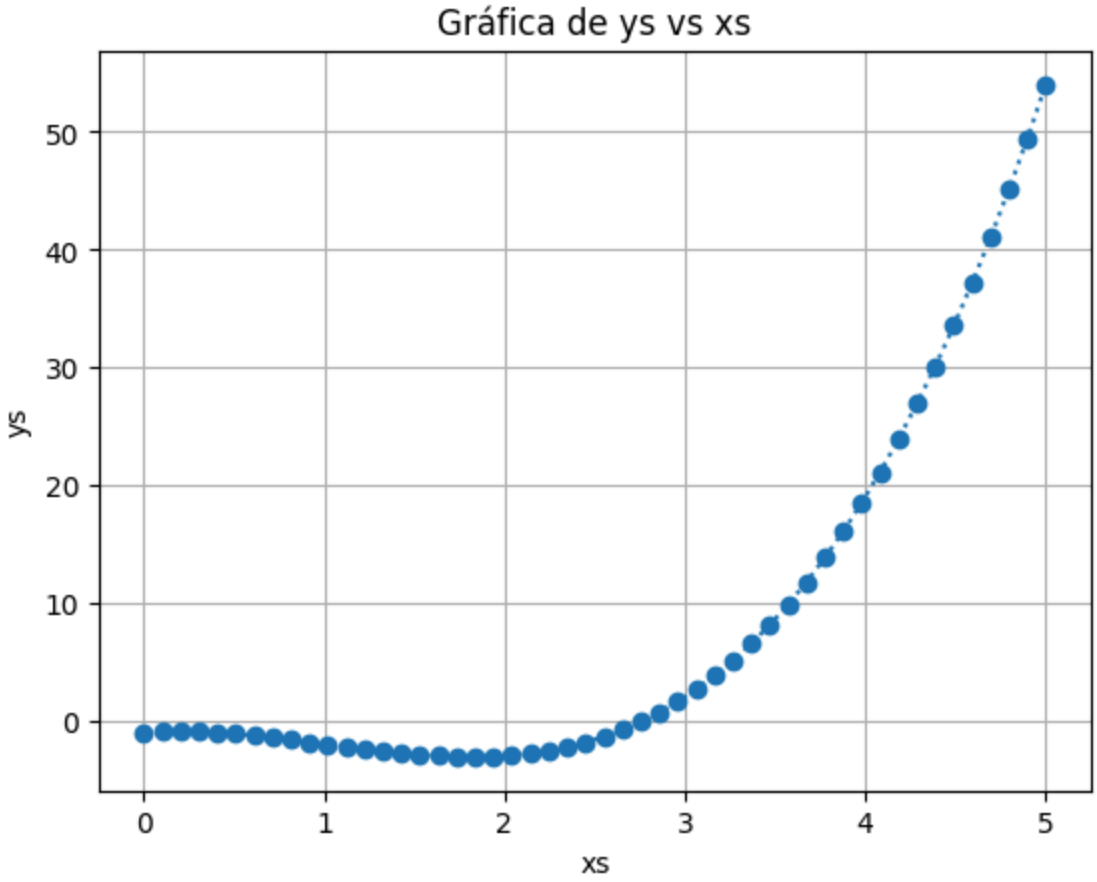
```
100.000000,  970099.00
67.004558,  287421.55
45.009926,  85151.64
30.350397,  25223.08
20.582860,  7468.63
14.079717,  2209.51
9.757836,  652.21
6.898519,  191.43
5.028702,  55.33
3.843681,  15.31
3.155971,  3.71
2.845416,  0.59
2.773143,  0.03
2.769303,  0.00
2.769292,  0.00
```

Out[13]: np.float64(2.7692923542386314)

Verificar visualmente

In [22]:
```python
import matplotlib.pyplot as plt

plt.plot(xs, ys, "o:")
plt.xlabel("xs")
plt.ylabel("ys")
plt.title("Gráfica de ys vs xs")
plt.grid(True)
plt.plot(f, 0, 'r*', markersize=20, label=f'Raíz: {f:.4f}')
plt.legend()
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 8
      6 plt.title("Gráfica de ys vs xs")
      7 plt.grid(True)
----> 8 plt.plot(f, 0, 'r*', markersize=20, label=f'Raíz: {f:.4f}')
      9 plt.legend()
     10 plt.show()

TypeError: unsupported format string passed to function.__format__
```



OSCILACIÓN ENTRE 0 Y 1

In [16]:
```
from scipy.optimize import newton

newton(f, x0=-10, fprime=lambda x: 3 * x**2 - 6 * x + 1)
```

-10.000000, -1311.00
-6.368421, -387.32
-3.960916, -114.17
-2.371519, -33.58
-1.325413, -9.92
-0.627655, -3.06
-0.113720, -1.15
0.556767, -1.20
-0.294347, -1.58
0.227717, -0.92
-4.119091, -125.91
-2.475710, -37.04
-1.394072, -10.93
-0.674503, -3.35
-0.152622, -1.23
0.464848, -1.08
-0.484418, -2.30
0.014894, -0.99
1.096611, -2.19
-0.015114, -1.02
0.915646, -1.83
-0.010182, -1.01
0.941855, -1.88
-0.004900, -1.00
0.971304, -1.94
-0.001213, -1.00
0.992766, -1.99
-0.000078, -1.00
0.999531, -2.00
-0.000000, -1.00
0.999998, -2.00
-0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00
1.000000, -2.00
0.000000, -1.00

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[16], line 3
      1 from scipy.optimize import newton
----> 3 newton(f, x0=-10, fprime=lambda x: 3 * x**2 - 6 * x + 1)

File c:\Users\Nathaly\AppData\Local\Programs\Python\Python314\Lib\site-packages\scipy\optimize\_zeros_py.py:392, in newton(func, x0, fprime, args, tol, maxiter, fprime2, x1, rtol, full_output, disp)
    390 if disp:
    391     msg = f"Failed to converge after {itr + 1} iterations, value is {p}."
--> 392     raise RuntimeError(msg)
    394 return _results_select(full_output, (p, funcalls, itr + 1, _ECONVERR), method)

RuntimeError: Failed to converge after 50 iterations, value is 1.0.
```

In [17]:
```python
import matplotlib.pyplot as plt
from scipy.optimize import newton

historial_x = []
historial_y = []

def f_registro(x):
    _y = x**3 - 3 * x**2 + x - 1
    historial_x.append(x)
    historial_y.append(_y)
    print(f"{x:10.5f}  {_y:15.5f}")
    return _y
```

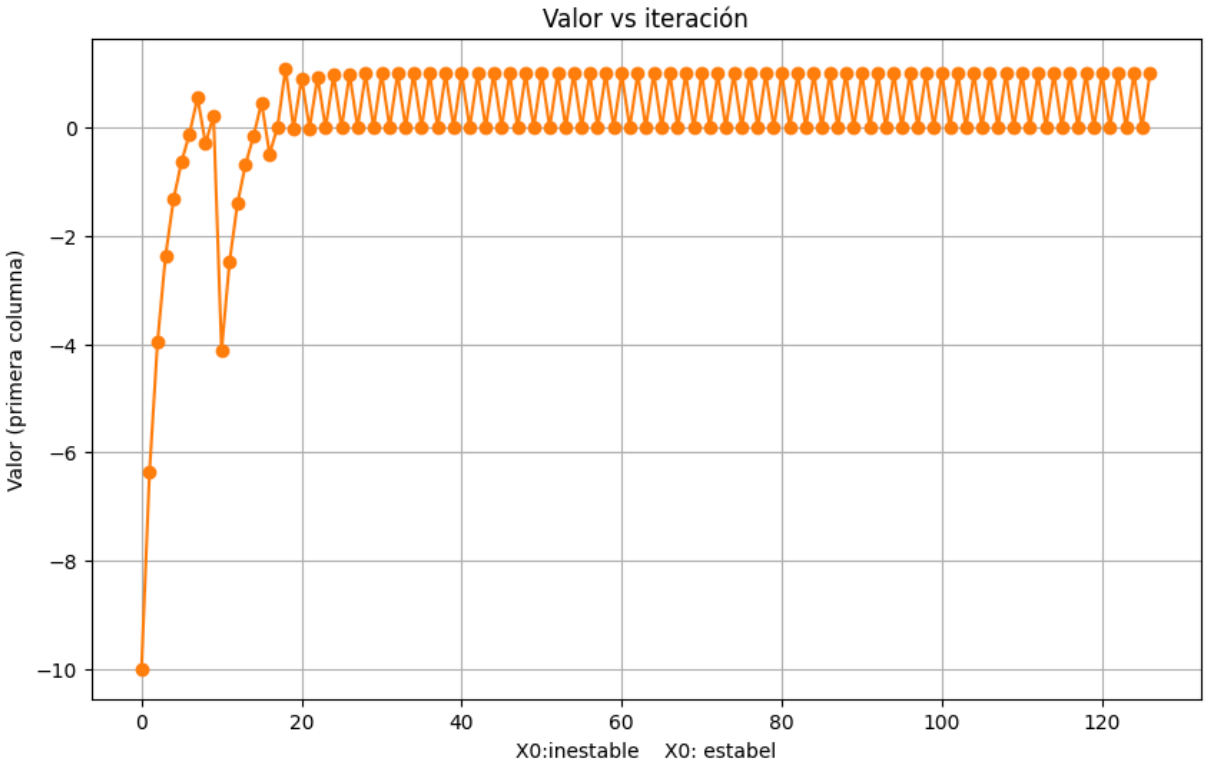
In [18]:
```
data_text = """-10.00000 -1311.00000
-6.36842 -387.32148
-3.96092 -114.16975
-2.37152 -33.58148
-1.32541 -9.92395
-0.62766 -3.05677
-0.11372 -1.15399
0.55677 -1.20061
-0.29435 -1.57977
0.22772 -0.91604
-4.11909 -125.90807
-2.47571 -37.03711
-1.39407 -10.93368
-0.67450 -3.34623
-0.15262 -1.22606
0.46485 -1.08296
-0.48442 -2.30207
0.01489 -0.98577
1.09661 -2.19232
-0.01511 -1.01580
0.91565 -1.83189
-0.01018 -1.01049
0.94186 -1.88391
-0.00490 -1.00497
0.97130 -1.94263
```

```
-0.00121 -1.00122
0.99277 -1.98553
-0.00008 -1.00008
0.99953 -1.99906
-0.00000 -1.00000
1.00000 -2.00000
-0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
```

```
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000
0.00000 -1.00000
1.00000 -2.00000"""

points = np.fromstring(data_text, sep=' ').reshape(-1, 2)
x_data = points[:, 0]
y_data = points[:, 1]
```

Con mas iteraciones

In [41]:
```
iteraciones = np.arange(len(points))
valores = [x for x, _ in points]

plt.figure(figsize=(10, 6))
plt.plot(iteraciones, valores, 'o-', color='C1', markersize=6, linewidth=1.5)
plt.xlabel('Iteración')
plt.ylabel('Valor (primera columna)')
plt.title('Valor vs iteración')
plt.xlabel('X0:inestable    X0: estabel')
plt.grid(True)
plt.show()
```



In [52]:
```
import matplotlib.pyplot as plt
from scipy.optimize import newton

historial_x = []
historial_y = []

def f(x):
    _y = x**3 - 3 * x**2 + x - 1
    historial_x.append(x)
    historial_y.append(_y)
    print(f"{x:10.5f}  {_y:15.5f}")
    return _y

def fder(x):
    return 3 * x**2 - 6 * x + 1

root = newton(f, 100, fprime=fder)
print(root)
```

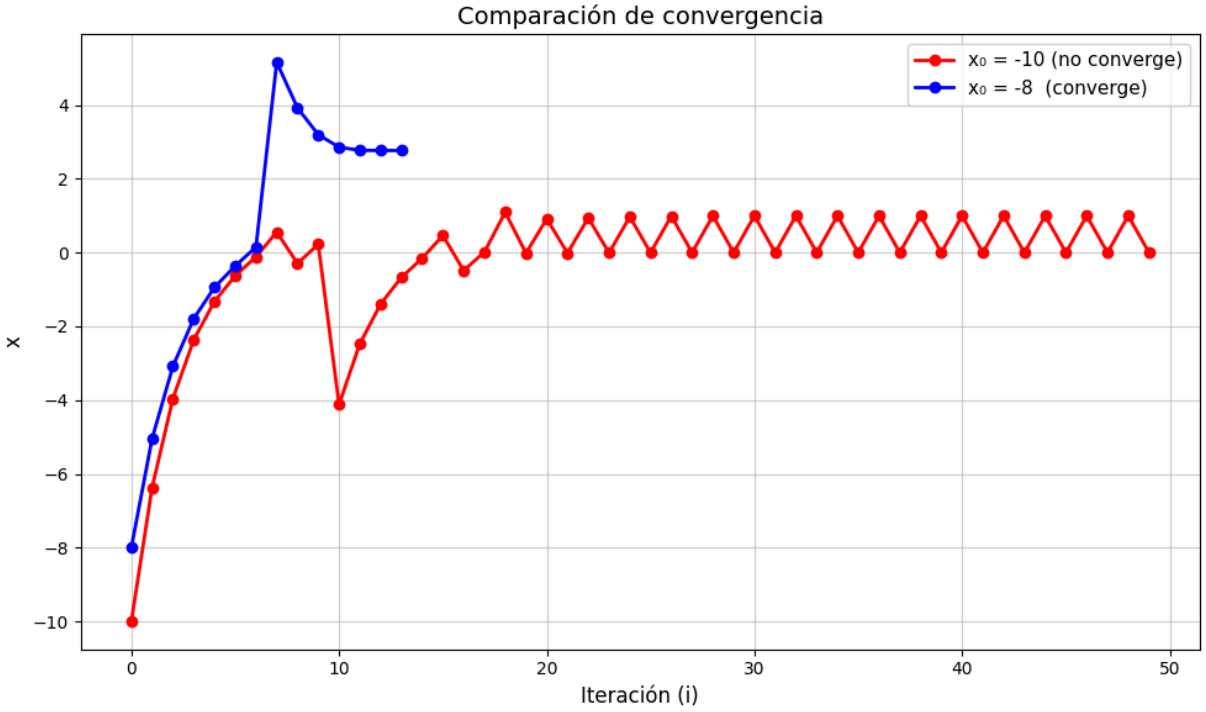
```
 100.00000     970099.00000
  67.00456     287421.55460
  45.00993      85151.64374
  30.35040      25223.07616
  20.58286       7468.63452
  14.07972       2209.50535
   9.75784        652.20763
   6.89852        191.42730
   5.02870         55.33019
   3.84368         15.30812
   3.15597          3.70946
   2.84542          0.59384
   2.77314          0.02854
   2.76930          0.00008
   2.76929          0.00000
2.7692923542386314
```

Gráfica de Comparación de convergencia

In [ ]:

```python
def f(x):
    return x**3 - 3*x**2 + x - 1

def df(x):
    return 3*x**2 - 6*x + 1


historial_x_10 = []

def f_registro_10(x):
    historial_x_10.append(x)
    return x**3 - 3*x**2 + x - 1

try:
    newton(f_registro_10, x0=-10, fprime=df, maxiter=50)
except RuntimeError:
    pass


historial_x_8 = []

def f_registro_8(x):
    historial_x_8.append(x)
    return x**3 - 3*x**2 + x - 1

try:
    newton(f_registro_8, x0=-8, fprime=df, maxiter=50)
except RuntimeError:
    pass


iter_10 = list(range(len(historial_x_10)))
iter_8 = list(range(len(historial_x_8)))

# Grafica
plt.figure(figsize=(10, 6))
```

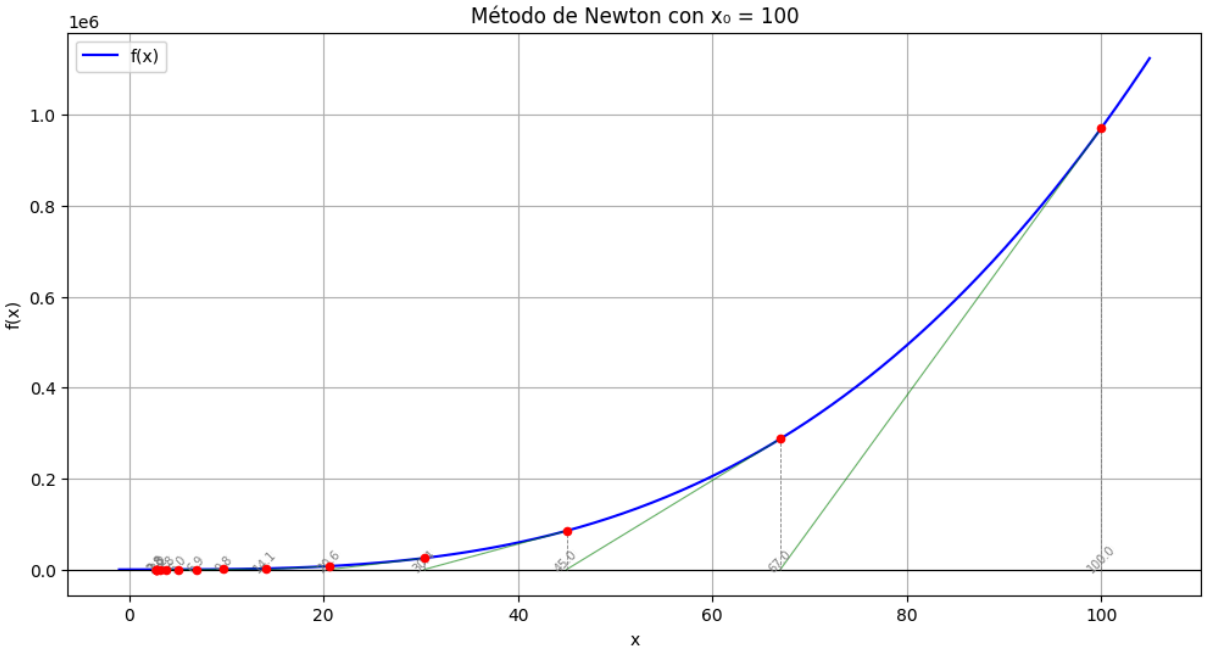
```
plt.plot(iter_10, historial_x_10, marker='o', color='red',
         linewidth=2, markersize=6, label="x₀ = -10 (no converge)")
plt.plot(iter_8, historial_x_8, marker='o', color='blue',
         linewidth=2, markersize=6, label="x₀ = -8  (converge)")

plt.xlabel("Iteración (i)", fontsize=12)
plt.ylabel("x", fontsize=12)
plt.title("Comparación de convergencia", fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.6)
plt.tight_layout()
plt.show()
```



Gráfica con la tangente

In [6]:
```
historial_x_100 = []
historial_y_100 = []

def df(x):
    return 3*x**2 - 6*x + 1

def f_registro_100(x):
    _y = x**3 - 3 * x**2 + x - 1
    historial_x_100.append(x)
    historial_y_100.append(_y)
    return _y

newton(f_registro_100, x0=100, fprime=df)

xs_wide = list(np.linspace(-1, max(historial_x_100) + 5, 500))
ys_wide = [x**3 - 3*x**2 + x - 1 for x in xs_wide]
```

```
plt.figure(figsize=(12, 6))
plt.plot(xs_wide, ys_wide, color='blue', label="f(x)")
plt.axhline(y=0, color='black', linewidth=0.8)

for i in range(len(historial_x_100) - 1):
    x_i = historial_x_100[i]
    y_i = historial_y_100[i]
    x_next = historial_x_100[i + 1]

    plt.plot([x_i, x_next], [y_i, 0], color='green', linewidth=0.8, alpha=0.6)
    plt.plot([x_i, x_i], [0, y_i], color='gray', linewidth=0.6, linestyle='--')
    plt.text(x_i, -5000, f"{x_i:.1f}", fontsize=7, ha='center', color='gray', rotat

plt.scatter(historial_x_100, historial_y_100, color='red', zorder=5, s=20)

plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Método de Newton con x₀ = 100")
plt.legend()
plt.grid(True)
plt.show()
```



In [ ]: